Handwritten digit recognition

Patrik Pusztai

April 2025

1 Important libraries

CV- computer vision
Tensorflow- used for machine learning

2 Introduction

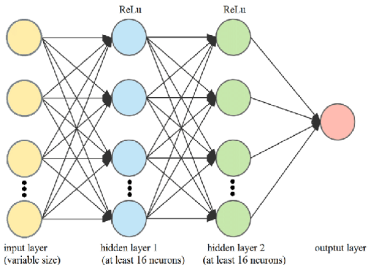
In this project we will use neural networks for recognizing handwritten digits with a high level of accuracy.

2.1 Important notions

The dataset we used is **mnist** which is a large database of handwritten digits. For the neural network we split this dataset into two parts: training data(used to train the model) and testing data(used to asses the model).

The first step is we create the tuples(xtrain, ytrain) and (xtest, ytest) in this case x is the pixel data(or the images themselves) and y is the classification(the number/the digit). We normalize the pixels which means we scale the images down such that the brightness of each pixel is between 0 and 1. We do this to make it easier for the neural network to do the calculations.

The neural network model we create is sequential. This means that the neurons are organized in layers that are stacked sequentially one after the other:



3 Creating the Neural Network

We start out with a flatten layer, we basically flatten the input shape such that it's not longer a grid of 28 by 28 but one big line if 28x28=784 pixels.This is important because every layer in a neural network does a bunch of matrix multiplications (aka dot products).Dense (fully connected) layers expect 1D vectors and they can't handle spatial layouts. The activation is 'ReLU'(Rectify Linear Unit). The Rectified Linear Unit (ReLU) is a popular activation function predominantly used in deep learning models. It is a piecewise linear function that will output the input directly if it is positive, otherwise, it will output zero. For the last layer we add the activation function softmax. Softmax makes sure that all the neurons add up to one and signals how likely that image is to be that digit. The way the neural network will work is: We have 10 digits , with 10 corresponding neurons. At the end of the analysis each neuron will contain a probability for a digit to be that answer.

4 Compile the Neural Network

To compile the neural network we must choose

1. A loss function: measures the difference between what the model predicated and what the correct answer was(it calculates how far off the guess was)
   Think of it as a scorecard: low score-good prediction, high score-bad prediction

2. An optimizer: uses the output from the loss function to adjust the weights and make the model do better in the future

3. The metrics to track during training

The flow is the following: the model makes a prediction -¿ the loss function says 'that was x amount wrong' , the optimizer updates weights to try and make the prediction better

5 Use the Neural Network

We created a directory called digits where we added a bunch of numbers. We read the files from the directory , invert them (they are not on the right by default) and turn the image into a numpy array. Afterwards we use the model to predict the digit. The function predict returns the activation of all the digit neurons so we must choose the digit neuron with the highest prediction.